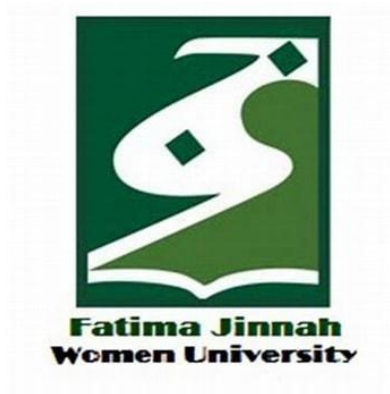


CLOUD COMPUTING

LAB NO# 09



SUBMITTED TO:

Sir Shohaib

SUBMITTED BY:

Name: Amina Noor

Reg No. 2023-BSE-007

Semester: V-A

CLOUD COMPUTING

LAB NO# 09

STEP 1: Install GitHub CLI on Your Local Machine (Windows)

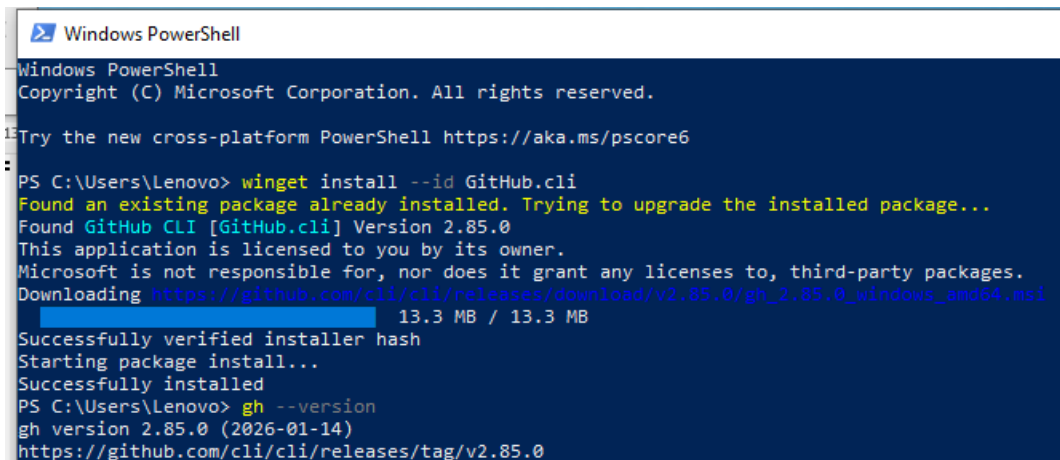
► Command:

winget install --id GitHub.cli

Wait for installation to complete.

✓ Verify:

gh --version



```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Try the new cross-platform PowerShell https://aka.ms/pscore6

PS C:\Users\Lenovo> winget install --id GitHub.cli
Found an existing package already installed. Trying to upgrade the installed package...
Found GitHub CLI [GitHub.cli] Version 2.85.0
This application is licensed to you by its owner.
Microsoft is not responsible for, nor does it grant any licenses to, third-party packages.
Downloading https://github.com/cli/cli/releases/download/v2.85.0/gh_2.85.0_windows_amd64.msi
13.3 MB / 13.3 MB
Successfully verified installer hash
Starting package install...
Successfully installed
PS C:\Users\Lenovo> gh --version
gh version 2.85.0 (2026-01-14)
https://github.com/cli/cli/releases/tag/v2.85.0
```

◆ STEP 2: Login to GitHub CLI (for Codespaces)

► Command:

gh auth login -s codespace

● Answer the prompts like this:

1. What account do you want to log into?
→ GitHub.com
2. Preferred protocol?
→ HTTPS
3. Authenticate GitHub CLI how?
→ Paste an authentication token

◆ STEP 3: Create GitHub Access Token (Browser)

When GH CLI opens browser:

1. Click **Generate new token (classic)**
2. Token name:
codespace-token
3. Expiration:
7 days (or 30 days)
4. **Select scopes (IMPORTANT):**
 - ☒ admin:org
 - ☒ codespace
 - ☒ repo
5. Click **Generate token**

◆ STEP 4: Paste Token into Terminal

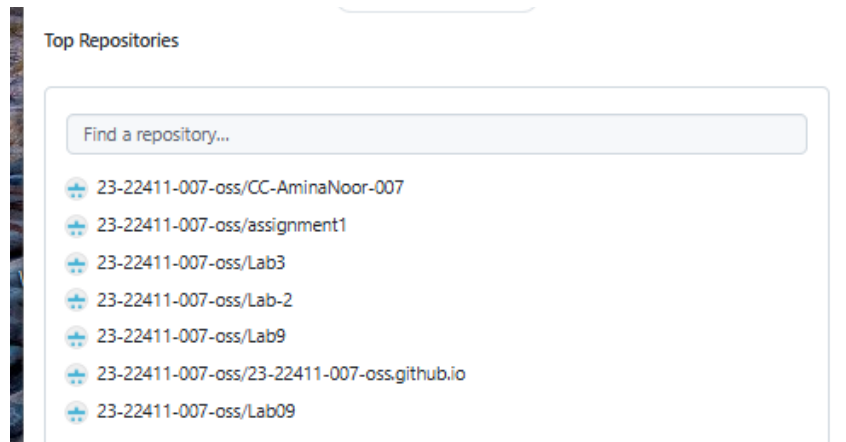
- Go back to terminal
- Paste token
- Press **Enter**

```
PS C:\Users\Lenovo> gh auth login -s codespace
? Where do you use GitHub? GitHub.com
? What is your preferred protocol for Git operations on this host? HTTPS
? Authenticate Git with your GitHub credentials? Yes
? How would you like to authenticate GitHub CLI? Paste an authentication token
Tip: you can generate a Personal Access Token here https://github.com/settings/tokens
The minimum required scopes are 'repo', 'read:org', 'workflow'.
? Paste your authentication token:
x Sorry, your reply was invalid: Value is required
? Paste your authentication token: *****
error validating token: HTTP 401: Bad credentials (https://api.github.com/)
Try authenticating with: gh auth login
PS C:\Users\Lenovo> gh auth logout
>> gh auth login -s codespace
? Logged out of github.com account 23-22411-007-oss
? Where do you use GitHub? GitHub.com
? What is your preferred protocol for Git operations on this host? HTTPS
? Authenticate Git with your GitHub credentials? Yes
? How would you like to authenticate GitHub CLI? Paste an authentication token
Tip: you can generate a Personal Access Token here https://github.com/settings/tokens
The minimum required scopes are 'repo', 'read:org', 'workflow'.
? Paste your authentication token: *****
- gh config set -h github.com git_protocol https
? Configured git protocol
? Logged in as 23-22411-007-oss
PS C:\Users\Lenovo>
```

◆ STEP 5: Create Lab9 Repository (If Not Created)

► Command:

gh repo create Lab9 --public



◆ STEP 6: List Existing Codespaces (Verification)

► Command:

gh codespace list

```
PS C:\Users\Lenovo> gh repo create Lab09 --public
Created repository 23-22411-007-oss/Lab09 on github.com
https://github.com/23-22411-007-oss/Lab09
PS C:\Users\Lenovo> gh codespace list
```

NAME	DISPLAY NAME	REPOSITORY	BRANCH	STATE	CREATED AT
upgraded-spork-7vj9w97gv7jxfrr59	upgraded spork	23-22411-007-oss/Ubuntu...	main	Shutdown	about 12 days ago
literate-xylophone-wr7x7vwr5xpc...	literate xylophone	23-22411-007-oss/Lab9	main*	Shutdown	about 4 days ago

```
PS C:\Users\Lenovo>
```

◆ STEP 7: Create a New Codespace

► Command:

gh codespace create --repo <your-github-username>/Lab9 --branch main --machine
basicLinux32gb

```
PS C:\Users\Lenovo\Lab09> git config --global user.name "23-22411-007-oss"
>> git config --global user.email "23-22411-007@se.fjwu.edu.pk"
PS C:\Users\Lenovo\Lab09> git config --global --list
user.name=23-22411-007-oss
user.email=23-22411-007@se.fjwu.edu.pk
PS C:\Users\Lenovo\Lab09> git add .
>> git commit -m "Initial commit for Codespaces"
>> git push origin main
>>
[main (root-commit) f66f90a] Initial commit for Codespaces
1 file changed, 0 insertions(+), 0 deletions(-)
create mode 100644 README.md
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Delta compression using up to 4 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 287 bytes | 143.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/23-22411-007-oss/Lab09.git
* [new branch] main -> main
PS C:\Users\Lenovo\Lab09> gh codespace create --repo 23-22411-007-oss/Lab09 --branch main --machine basicLinux32gb
A codespace is being created for this repository. It is paid for by 23-22411-007-oss
refactored-broccoli-g4vq6qxg45jjfvpvgv
PS C:\Users\Lenovo\Lab09>
```

STEP 8: Connect to Codespace (LOCAL PowerShell)

gh codespace list

```
refactored-broccoli-g4vq6qyg45jjfvpv
PS C:\Users\Lenovo\Lab09> gh codespace list


| NAME                                | DISPLAY NAME        | REPOSITORY              | BRANCH | STATE     | CREATED AT         |
|-------------------------------------|---------------------|-------------------------|--------|-----------|--------------------|
| upgraded-spork-7vj9w97gv7jxfrr59    | upgraded spork      | 23-22411-007-oss/Ubu... | main   | Shutdown  | about 12 days ago  |
| literate-xylophone-wrq7x7vwr5xpc... | literate xylophone  | 23-22411-007-oss/Lab9   | main*  | Shutdown  | about 4 days ago   |
| refactored-broccoli-g4vq6qyg45jj... | refactored broccoli | 23-22411-007-oss/Lab09  | main   | Available | about 1 minute ago |


PS C:\Users\Lenovo\Lab09>
```

Now connect:

gh codespace ssh -c refactored broccoli

```
PS C:\Users\Lenovo\Lab09> gh codespace ssh -c refactored-broccoli-g4vq6qyg45jjfvpv
Welcome to Ubuntu 24.04.3 LTS (GNU/Linux 6.8.0-1030-azure x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

@23-22411-007-oss /workspaces/Lab09 (main) $
```

◆ STEP 9: Verify You Are INSIDE Codespace

You should see:

@codespaces → /workspaces/Lab9 \$

Run:

uname -a

```
@23-22411-007-oss /workspaces/Lab09 (main) $ uname -a
linux codespaces-70f5e0 6.8.0-1030-azure #35~22.04.1-Ubuntu SMP Mon May 26 18:08:30 UTC 2025 x86_64 x86_64 x86_64 GNU/L
inux
@23-22411-007-oss /workspaces/Lab09 (main) $
```

🔧 Task 2 — Install & Configure AWS CLI (BABY STEPS)

◆ STEP 1: Download AWS CLI (Linux)

► Command (copy-paste):

```
curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o "awscliv2.zip"
```

```

@23-22411-007-oss @ /workspaces/Lab09 (main) $ curl "http://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o "awscli
v2.zip"
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left   Speed
100  167  100  167    0     0   3834      0  --:--:-- --:--:-- --:--:--   3883
@23-22411-007-oss @ /workspaces/Lab09 (main) $

```

⌚ Wait until download finishes.

◆ STEP 2: Unzip AWS CLI Package

► Command:

```
unzip awscliv2.zip
```

If unzip is missing (rare), run:

```
sudo apt update && sudo apt install unzip -y
```

Then unzip again.

◆ STEP 3: Install AWS CLI

► Command:

```
sudo ./aws/install
```

```

Initiating. aws/dist/wheel-0.43.1/dist-info/RECORD
@23-22411-007-oss @ /workspaces/Lab09 (main) $ sudo ./aws/install
You can now run: /usr/local/bin/aws --version

```

◆ STEP 4: Verify AWS CLI Installation

► Command:

```
aws --version
```

```

@23-22411-007-oss @ /workspaces/Lab09 (main) $ aws --version
aws-cli/2.33.5 Python/3.13.11 Linux/6.8.0-1030-azure exe/x86_64.ubuntu.24
@23-22411-007-oss @ /workspaces/Lab09 (main) $

```

Windows PowerShell

```
inflating: aws/dist/awscli/botocore/data/sesv2/2019-09-27/paginators-1.json
inflating: aws/dist/awscli/botocore/data/sesv2/2019-09-27/service-2.json
  creating: aws/dist/awscli/botocore/.changes/next-release/
inflating: aws/dist/awscli/botocore/.changes/next-release/api-change-connect-59117.json
inflating: aws/dist/awscli/topics/s3-faq.rst
inflating: aws/dist/awscli/topics/config-vars.rst
inflating: aws/dist/awscli/topics/ddb-expressions.rst
inflating: aws/dist/awscli/topics/return-codes.rst
inflating: aws/dist/awscli/topics/s3-config.rst
inflating: aws/dist/awscli/topics/topic-tags.json
inflating: aws/dist/awscli/topics/s3-case-insensitivity.rst
inflating: aws/dist/awscli/data/metadata.json
inflating: aws/dist/awscli/data/ac.index
inflating: aws/dist/awscli/data/cli.json
  creating: aws/dist/awscli/customizations/sso/
  creating: aws/dist/awscli/customizations/wizard/
  creating: aws/dist/awscli/customizations/wizard/wizards/
  creating: aws/dist/awscli/customizations/wizard/wizards/configure/
  creating: aws/dist/awscli/customizations/wizard/wizards/dynamodb/
  creating: aws/dist/awscli/customizations/wizard/wizards/events/
  creating: aws/dist/awscli/customizations/wizard/wizards/iam/
  creating: aws/dist/awscli/customizations/wizard/wizards/lambda/
inflating: aws/dist/awscli/customizations/wizard/wizards/iam/new-role.yml
inflating: aws/dist/awscli/customizations/wizard/wizards/events/new-rule.yml
inflating: aws/dist/awscli/customizations/wizard/wizards/lambda/new-function.yml
inflating: aws/dist/awscli/customizations/wizard/wizards/configure/_main.yml
inflating: aws/dist/awscli/customizations/wizard/wizards/dynamodb/new-table.yml
inflating: aws/dist/awscli/customizations/sso/index.html
  creating: aws/dist/prompt_toolkit-3.0.51.dist-info/licenses/
inflating: aws/dist/prompt_toolkit-3.0.51.dist-info/INSTALLER
inflating: aws/dist/prompt_toolkit-3.0.51.dist-info/METADATA
inflating: aws/dist/prompt_toolkit-3.0.51.dist-info/WHEEL
inflating: aws/dist/prompt_toolkit-3.0.51.dist-info/top_level.txt
inflating: aws/dist/prompt_toolkit-3.0.51.dist-info/RECORD
inflating: aws/dist/prompt_toolkit-3.0.51.dist-info/licenses/AUTHORS.rst
inflating: aws/dist/prompt_toolkit-3.0.51.dist-info/licenses/LICENSE
inflating: aws/dist/wheel-0.45.1.dist-info/entry_points.txt
inflating: aws/dist/wheel-0.45.1.dist-info/LICENSE.txt
inflating: aws/dist/wheel-0.45.1.dist-info/METADATA
inflating: aws/dist/wheel-0.45.1.dist-info/REQUESTED
inflating: aws/dist/wheel-0.45.1.dist-info/direct_url.json
inflating: aws/dist/wheel-0.45.1.dist-info/INSTALLER
inflating: aws/dist/wheel-0.45.1.dist-info/WHEEL
inflating: aws/dist/wheel-0.45.1.dist-info/RECORD
023-22411-007-oss @ /workspaces/Lab09 (main) $ sudo ./aws/install
You can now run: /usr/local/bin/aws --version
023-22411-007-oss @ /workspaces/Lab09 (main) $ aws --version
aws-cli/2.33.5 Python/3.13.11 Linux/6.8.0-1030-azure exe/x86_64.ubuntu.24
023-22411-007-oss @ /workspaces/Lab09 (main) $
```

◆ STEP 5: Configure AWS CLI

► Command:

aws configure

◆ STEP 6: Verify AWS Config Files

► Command:

cat ~/.aws/credentials

Then:

cat ~/.aws/config

```

• @23-22411-007-oss →/workspaces/Lab09 (main) $ aws configure
  AWS Access Key ID [None]: AKIATFX3NIVRN2VJ5CVQ
  AWS Secret Access Key [None]: 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

  Default output format [None]: json
• @23-22411-007-oss →/workspaces/Lab09 (main) $ cat ~/.aws/credentials
[default]
aws_access_key_id = AKIATFX3NIVRN2VJ5CVQ
aws_secret_access_key = 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
• @23-22411-007-oss →/workspaces/Lab09 (main) $ cat ~/.aws/config
[default]
output = json
○ @23-22411-007-oss →/workspaces/Lab09 (main) $ █

```

◆ STEP 7: Test AWS Connectivity (MOST IMPORTANT)

► Command:

aws sts get-caller-identity

```

• @23-22411-007-oss →/workspaces/Lab09 (main) $ aws sts get-caller-identity
{
  "UserId": "AIDATFX3NIVRBKXYLMQ05",
  "Account": "218496845154",
  "Arn": "arn:aws:iam::218496845154:user/lab9"
}
○ @23-22411-007-oss →/workspaces/Lab09 (main) $ █

```

🔑 Task 3 — EC2 Security Group Setup (Step-by-Step)

Goal: Create a security group, add SSH and HTTP ingress rules from your Codespace IP, verify them.

◆ STEP 1: Identify VPC ID

Before creating a security group, you need your **VPC ID**.

► Command:

aws ec2 describe-vpcs --query "Vpcs[0].VpcId" --output text

```

• @23-22411-007-oss →/workspaces/Lab09 (main) $ aws ec2 describe-vpcs --query "Vpcs[0].VpcId" --output text
vpc-082ef0dc0c9f1d07d

```

◆ STEP 2: Create Security Group

► Command:

```
aws ec2 create-security-group --group-name 'MySecurityGroup' \  
  
--description 'My Security Group' \  
  
--vpc-id 'vpc-0abc1234def567890'
```

```
@23-22411-007-oss →/workspaces/Lab09 (main) $ aws ec2 create-security-group --group-name 'MySecurityGroup' --description 'My Security Group' --vpc-id 'vpc-0abc1234def567890' aws ec2 create-security-group --group-name 'MySecurityGroup' \  
--description 'My Security Group' \  
--vpc-id 'vpc-082ef0dc0c9f1d07d'  
  
usage: aws [options] <command> <subcommand> [<subcommand> ...] [parameters]  
To see help text, you can run:  
  
aws help  
aws <command> help  
aws <command> <subcommand> help  
  
Unknown options: create-security-group, ec2
```

◆ STEP 3: Inspect Security Group Before Adding Rules

► Command:

```
aws ec2 describe-security-groups --group-ids sg-0a1b2c3d4e5f67890
```

```
@23-22411-007-oss →/workspaces/Lab09 (main) $ aws ec2 describe-security-groups --group-ids sg-0aea5ee66ca09fe98  
{  
  "SecurityGroups": [  
    {  
      "GroupId": "sg-0aea5ee66ca09fe98",  
      "IpPermissionsEgress": [  
        {  
          "IpProtocol": "-1",  
          "UserIdGroupPairs": [],  
          "IpRanges": [  
            {  
              "CidrIp": "0.0.0.0/0"  
            }  
          ],  
          "Ipv6Ranges": [],  
          "PrefixListIds": []  
        }  
      ],  
      "VpcId": "vpc-082ef0dc0c9f1d07d",  
      "SecurityGroupArn": "arn:aws:ec2:me-central-1:218496845154:security-group/sg-0aea5ee66ca09fe98",  
      "OwnerId": "218496845154",  
      "GroupName": "default",  
      "Description": "default VPC security group",  
      "IpPermissions": [  
        {  
          "IpProtocol": "-1",  
          "UserIdGroupPairs": [  
            {  
              "UserId": "218496845154",  
              "GroupId": "sg-0aea5ee66ca09fe98"  
            }  
          ]  
        }  
      ]  
    }  
  ]  
}
```



```

@23-22411-007-oss →/workspaces/Lab09 (main) $ aws ec2 authorize-security-group-ingress \
--group-id sg-0aea5ee66ca09fe98 \
--protocol tcp \
--port 22 \
--cidr 4.240.18.229/32
{
  "Return": true,
  "SecurityGroupRules": [
    {
      "SecurityGroupRuleId": "sgr-0309c1d3b52fc4665",
      "GroupId": "sg-0aea5ee66ca09fe98",
      "GroupOwnerId": "218496845154",
      "IsEgress": false,
      "IpProtocol": "tcp",
      "FromPort": 22,
      "ToPort": 22,
      "CidrIpv4": "4.240.18.229/32",
      "SecurityGroupRuleArn": "arn:aws:ec2:me-central-1:218496845154:security-group-rule/sgr-0309c1d3b52fc4665"
    }
  ]
}
@23-22411-007-oss →/workspaces/Lab09 (main) $

```

◆ STEP 6: Verify SSH Rule Added

► Command:

aws ec2 describe-security-groups --group-ids sg-0aea5ee66ca09fe98

```

@23-22411-007-oss →/workspaces/Lab09 (main) $ aws ec2 describe-security-groups --group-ids sg-0aea5ee66ca09fe98
{
  "SecurityGroups": [
    {
      "GroupId": "sg-0aea5ee66ca09fe98",
      "IpPermissionsEgress": [
        {
          "IpProtocol": "-1",
          "UserIdGroupPairs": [],
          "IpRanges": [
            {
              "CidrIp": "0.0.0.0/0"
            }
          ],
          "Ipv6Ranges": [],
          "PrefixListIds": []
        }
      ],
      "VpcId": "vpc-082ef0dc0c9f1d07d",
      "SecurityGroupArn": "arn:aws:ec2:me-central-1:218496845154:security-group/sg-0aea5ee66ca09fe98",
      "OwnerId": "218496845154",
      "GroupName": "default",
      "Description": "default VPC security group",
      "IpPermissions": [
        {
          "IpProtocol": "-1",
          "UserIdGroupPairs": [
            {
              "UserId": "218496845154",
              "GroupId": "sg-0aea5ee66ca09fe98"
            }
          ]
        }
      ]
    }
  ]
}

```

◆ STEP 7: Add HTTP Rule (Port 80) via ip-permissions JSON

► Command:

aws ec2 authorize-security-group-ingress \

```
--group-id sg-0aea5ee66ca09fe98\  
  
--ip-permissions '{"FromPort":80,"ToPort":80,"IpProtocol":"tcp","IpRanges":[{"CidrIp":  
4.240.18.229/32}]]}'
```

- Replace 203.0.113.25 with your Codespace IP

```
@23-22411-007-oss →/workspaces/Lab09 (main) $ aws ec2 authorize-security-group-ingress \  
--group-id sg-0aea5ee66ca09fe98 \  
--ip-permissions '{"FromPort":80,"ToPort":80,"IpProtocol":"tcp","IpRanges":[{"CidrIp":"4.240.18.229/32}]]}'  
{  
  "Return": true,  
  "SecurityGroupRules": [  
    {  
      "SecurityGroupRuleId": "sgr-005dae5710d2fde70",  
      "GroupId": "sg-0aea5ee66ca09fe98",  
      "GroupOwnerId": "218496845154",  
      "IsEgress": false,  
      "IpProtocol": "tcp",  
      "FromPort": 80,  
      "ToPort": 80,  
      "CidrIpv4": "4.240.18.229/32",  
      "SecurityGroupRuleArn": "arn:aws:ec2:me-central-1:218496845154:security-group-rule/sgr-005dae5710d2fde70"  
    }  
  ]  
}
```

◆ STEP 8: Verify HTTP Rule Added

► Command:

```
aws ec2 describe-security-groups --group-ids sg-0a1b2c3d4e5f67890
```

```
@23-22411-007-oss →/workspaces/Lab09 (main) $ aws ec2 describe-security-groups --group-ids sg-0aea5ee66ca09fe98  
{  
  "SecurityGroups": [  
    {  
      "GroupId": "sg-0aea5ee66ca09fe98",  
      "IpPermissionsEgress": [  
        {  
          "IpProtocol": "-1",  
          "UserIdGroupPairs": [],  
          "IpRanges": [  
            {  
              "CidrIp": "0.0.0.0/0"  
            }  
          ],  
          "Ipv6Ranges": [],  
          "PrefixListIds": []  
        }  
      ],  
      "VpcId": "vpc-082ef0dc9f1d07d",  
      "SecurityGroupArn": "arn:aws:ec2:me-central-1:218496845154:security-group/sg-0aea5ee66ca09fe98",  
      "OwnerId": "218496845154",  
      "GroupName": "default",  
      "Description": "default VPC security group",  
      "IpPermissions": [  
        {  
          "IpProtocol": "tcp",  
          "FromPort": 80,  
          "ToPort": 80,  
          "UserIdGroupPairs": [],  
          "IpRanges": [  
            {  
              "CidrIp": "4.240.18.229/32"  
            }  
          ],  
          "PrefixListIds": []  
        }  
      ]  
    }  
  ]  
}
```

◆ STEP 9: Final Check — All Rules

Optional but recommended:

aws ec2 describe-security-groups --group-ids sg-0a1b2c3d4e5f67890

- Verify both rules still exist

```
@23-22411-007-oss →/workspaces/Lab09 (main) $ aws ec2 describe-security-groups --group-ids sg-0aea5ee66ca09fe98
{
  "SecurityGroups": [
    {
      "GroupId": "sg-0aea5ee66ca09fe98",
      "IpPermissionsEgress": [
        {
          "IpProtocol": "-1",
          "UserIdGroupPairs": [],
          "IpRanges": [
            {
              "CidrIp": "0.0.0.0/0"
            }
          ],
          "Ipv6Ranges": [],
          "PrefixListIds": []
        }
      ],
      "VpcId": "vpc-082ef0dc0c9f1d07d",
      "SecurityGroupArn": "arn:aws:ec2:me-central-1:218496845154:security-group/sg-0aea5ee66ca09fe98",
      "OwnerId": "218496845154",
      "GroupName": "default",
      "Description": "default VPC security group",
      "IpPermissions": [
        {
          "IpProtocol": "tcp",
          "FromPort": 80,
          "ToPort": 80,
          "UserIdGroupPairs": [],
          "IpRanges": [

```

Task 4 — Create a key pair, describe key pairs, and launch EC2 instance

Step 1 — Create an ED25519 Key Pair

Run this command in your Codespace terminal:

```
aws ec2 create-key-pair \
  --key-name MyED25519Key \
  --key-type ed25519 \
  --key-format pem \
  --query 'KeyMaterial' \
  --output text > MyED25519Key.pem
```

```
IpRanges : [
• @23-22411-007-oss →/workspaces/Lab09 (main) $ aws ec2 create-key-pair \
  --key-name MyED25519Key \
  --key-type ed25519 \
  --key-format pem \
  --query 'KeyMaterial' \
  --output text > MyED25519Key.pem
```

Next: Check that the file is created:

ls -l MyED25519Key.pem

```
Output text / MyED25519Key.pem
• @23-22411-007-oss →/workspaces/Lab09 (main) $ ls -l MyED25519Key.pem
-rw-rw-rw- 1 codespace codespace 388 Jan 23 09:23 MyED25519Key.pem
```

Step 2 — View Created Key Pairs

aws ec2 describe-key-pairs

```
• @23-22411-007-oss →/workspaces/Lab09 (main) $ aws ec2 describe-key-pairs
{
  "KeyPairs": [
    {
      "KeyId": "key-0343975126529bd03",
      "KeyType": "ed25519",
      "Tags": [],
      "CreateTime": "2026-01-23T09:23:31.955000+00:00",
      "KeyName": "MyED25519Key",
      "KeyFingerprint": "SoC2FM2pzlg2x0u+nojr5vzpiYntAAO2OMPkz5E8H6Q="
    }
  ]
}
• @23-22411-007-oss →/workspaces/Lab09 (main) $
```

Step 3 — (Optional) Delete Key Pair

To show how deletion works (do **not** delete if you plan to use it):

aws ec2 delete-key-pair --key-name MyED25519Key

```
• @23-22411-007-oss →/workspaces/Lab09 (main) $ aws ec2 delete-key-pair --key-name MyED25519Key
{
  "Return": true,
  "KeyId": "key-0343975126529bd03"
}
• @23-22411-007-oss →/workspaces/Lab09 (main) $
```

Step 4 — Launch an EC2 Instance

Example command (replace IDs with your AMI, security group, and subnet):

aws ec2 run-instances \

--image-id ami-05e66df2bafcb7dea \

--count 1 \

--instance-type t3.micro \

--key-name MyED25519Key \

```
--security-group-ids sg-EXAMPLE1234567890 \
```

```
--subnet-id subnet-EXAMPLE1234567890 \
```

```
--tag-specifications "ResourceType=instance,Tags=[{Key=Name,Value=MyServer}]"
```

```
@23-22411-007-oss →/workspaces/Lab09 (main) $ aws ec2 run-instances \
--image-id ami-0f846b83494ddb084 \
--count 1 \
--instance-type t3.micro \
--key-name MyED25519Key \
--security-group-ids sg-0aea5ee66ca09fe98 \
--subnet-id subnet-0ed9f14bcd5dfebb3 \
--tag-specifications "ResourceType=instance,Tags=[{Key=Name,Value=MyServer}]"
{
  "ReservationId": "r-00c9fea882f9b15e6",
  "OwnerId": "218496845154",
  "Groups": [],
  "Instances": [
    {
      "Architecture": "x86_64",
      "BlockDeviceMappings": [],
      "ClientToken": "f791399e-6d3e-4769-bda0-5c821342eeb3",
      "EbsOptimized": false,
      "EnaSupport": true,
      "Hypervisor": "xen",
      "NetworkInterfaces": [
        {
          "Attachment": {
            "AttachTime": "2026-01-23T09:48:40+00:00",
            "AttachmentId": "eni-attach-0282a5cece775d438",
            "DeleteOnTermination": true,
            "DeviceIndex": 0,
            "Status": "attaching",
            "NetworkCardIndex": 0
          }
        }
      ]
    }
  ]
}
```

Step 5 — Get Public IP Address

```
aws ec2 describe-instances \
```

```
--query "Reservations[*].Instances[*].[InstanceId,PublicIpAddress]" \
```

```
--output table
```

```
@23-22411-007-oss →/workspaces/Lab09 (main) $ aws ec2 describe-instances \
--query "Reservations[*].Instances[*].[InstanceId,PublicIpAddress,State.Name]" \
--output table
-----+-----+-----+
|                               DescribeInstances                               |
+-----+-----+-----+
| i-093cd8ee90f34185b | None | shutting-down |
| i-03aca059942e3c1e5 | 3.28.45.180 | running |
+-----+-----+-----+
```

Step 6 — SSH into Your EC2 Instance

1. Fix Permissions

```

@g23-22411-007-oss → /workspaces/Lab09 (main) $ ssh -i MyED25519Key.pem ec2-user@3.28.45.180

[ec2-user@ip-172-31-0-192 ~]$ sudo yum update -y
Loaded plugins: extras_suggestions, langpacks, priorities, update-motd
amzn2-core                                     | 3.6 kB  00:00:00
amzn2extra-docker                             | 2.9 kB  00:00:00
(1/5): amzn2-core/2/x86_64/group_gz          | 2.7 kB  00:00:00
(2/5): amzn2-core/2/x86_64/updateinfo        | 1.2 MB  00:00:00
(3/5): amzn2extra-docker/2/x86_64/primary.db | 162 kB  00:00:00
(4/5): amzn2extra-docker/2/x86_64/updateinfo | 30 kB   00:00:00
(5/5): amzn2-core/2/x86_64/primary.db        | 83 MB   00:00:01
Resolving Dependencies
--> Running transaction check
--> Package amazon-ssm-agent.x86_64 0:3.3.3050.0-1.amzn2 will be updated
--> Package amazon-ssm-agent.x86_64 0:3.3.3572.0-1.amzn2 will be an update
--> Package aws-cfn-bootstrap.noarch 0:2.0-36.amzn2 will be updated
--> Package aws-cfn-bootstrap.noarch 0:2.0-38.amzn2 will be an update
--> Package ec2-hibinit-agent.noarch 0:1.0.10-0.amzn2 will be updated
--> Package ec2-hibinit-agent.noarch 0:1.0.10-1.amzn2 will be an update
--> Package glib2.x86_64 0:2.56.1-9.amzn2.0.12 will be updated
--> Package glib2.x86_64 0:2.56.1-9.amzn2.0.13 will be an update
--> Package gnupg2.x86_64 0:2.0.22-5.amzn2.0.5 will be updated
--> Package gnupg2.x86_64 0:2.0.22-5.amzn2.0.6 will be an update
--> Package grub2.x86_64 1:2.06-14.amzn2.0.6 will be obsoleted
--> Package grub2.x86_64 1:2.06-14.amzn2.0.7 will be obsoleting
--> Package grub2-common.noarch 1:2.06-14.amzn2.0.6 will be updated
--> Package grub2-common.noarch 1:2.06-14.amzn2.0.7 will be an update
--> Package grub2-efi-x64-ec2.x86_64 1:2.06-14.amzn2.0.6 will be updated
--> Package grub2-efi-x64-ec2.x86_64 1:2.06-14.amzn2.0.7 will be an update
--> Package grub2-pc.x86_64 1:2.06-14.amzn2.0.6 will be updated

```

2. Check instance info:

hostname

whoami

curl ifconfig.me

```
@23-22411-007-oss →/workspaces/Lab09 (main) $ ssh -i MyED25519Key.pem ec2-user@3.28.45.180
python-urllib3.noarch 0:1.25.9-1.amzn2.0.8          python3.x86_64 0:3.7.16-1.amzn2.0.22
python3-libs.x86_64 0:3.7.16-1.amzn2.0.22          python3-pip.noarch 0:20.2.2-1.amzn2.0.14
rsync.x86_64 0:3.1.2-11.amzn2.0.6                  systemtap-runtime.x86_64 0:4.5-1.amzn2.0.3
tzdata.noarch 0:2025c-1.amzn2.0.1                  util-linux.x86_64 0:2.30.2-2.amzn2.0.13

Replaced:
grub2.x86_64 1:2.06-14.amzn2.0.6                    grub2-tools.x86_64 1:2.06-14.amzn2.0.6

Complete!
[ec2-user@ip-172-31-0-192 ~]$ hostname
ip-172-31-0-192.me-central-1.compute.internal
[ec2-user@ip-172-31-0-192 ~]$ whoami
ec2-user
[ec2-user@ip-172-31-0-192 ~]$ curl ifconfig.me
3.28.45.180[ec2-user@ip-172-31-0-192 ~]$ ping google.com
```

3. Test network connectivity:

ping google.com

```
@23-22411-007-oss →/workspaces/Lab09 (main) $ ssh -i MyED25519Key.pem ec2-user@3.28.45.180
3.28.45.180[ec2-user@ip-172-31-0-192 ~]$ ping google.com
PING google.com (142.250.203.14) 56(84) bytes of data:
64 bytes from pnfjra-ak-in-f14.1e100.net (142.250.203.14): icmp_seq=1 ttl=118 time=2.73 ms
64 bytes from pnfjra-ak-in-f14.1e100.net (142.250.203.14): icmp_seq=2 ttl=118 time=2.67 ms
64 bytes from pnfjra-ak-in-f14.1e100.net (142.250.203.14): icmp_seq=3 ttl=118 time=2.69 ms
64 bytes from pnfjra-ak-in-f14.1e100.net (142.250.203.14): icmp_seq=4 ttl=118 time=2.86 ms
64 bytes from pnfjra-ak-in-f14.1e100.net (142.250.203.14): icmp_seq=5 ttl=118 time=2.71 ms
64 bytes from pnfjra-ak-in-f14.1e100.net (142.250.203.14): icmp_seq=6 ttl=118 time=2.71 ms
64 bytes from pnfjra-ak-in-f14.1e100.net (142.250.203.14): icmp_seq=7 ttl=118 time=2.68 ms
64 bytes from pnfjra-ak-in-f14.1e100.net (142.250.203.14): icmp_seq=8 ttl=118 time=2.72 ms
64 bytes from pnfjra-ak-in-f14.1e100.net (142.250.203.14): icmp_seq=9 ttl=118 time=2.69 ms
64 bytes from pnfjra-ak-in-f14.1e100.net (142.250.203.14): icmp_seq=10 ttl=118 time=2.68 ms
64 bytes from pnfjra-ak-in-f14.1e100.net (142.250.203.14): icmp_seq=11 ttl=118 time=2.74 ms
64 bytes from pnfjra-ak-in-f14.1e100.net (142.250.203.14): icmp_seq=12 ttl=118 time=2.67 ms
64 bytes from pnfjra-ak-in-f14.1e100.net (142.250.203.14): icmp_seq=13 ttl=118 time=2.69 ms
64 bytes from pnfjra-ak-in-f14.1e100.net (142.250.203.14): icmp_seq=14 ttl=118 time=2.68 ms
64 bytes from pnfjra-ak-in-f14.1e100.net (142.250.203.14): icmp_seq=15 ttl=118 time=2.67 ms
64 bytes from pnfjra-ak-in-f14.1e100.net (142.250.203.14): icmp_seq=16 ttl=118 time=2.69 ms
64 bytes from pnfjra-ak-in-f14.1e100.net (142.250.203.14): icmp_seq=17 ttl=118 time=2.67 ms
64 bytes from pnfjra-ak-in-f14.1e100.net (142.250.203.14): icmp_seq=18 ttl=118 time=2.71 ms
64 bytes from pnfjra-ak-in-f14.1e100.net (142.250.203.14): icmp_seq=19 ttl=118 time=2.68 ms
64 bytes from pnfjra-ak-in-f14.1e100.net (142.250.203.14): icmp_seq=20 ttl=118 time=2.68 ms
64 bytes from pnfjra-ak-in-f14.1e100.net (142.250.203.14): icmp_seq=21 ttl=118 time=2.68 ms
64 bytes from pnfjra-ak-in-f14.1e100.net (142.250.203.14): icmp_seq=22 ttl=118 time=2.68 ms
64 bytes from pnfjra-ak-in-f14.1e100.net (142.250.203.14): icmp_seq=23 ttl=118 time=2.68 ms
64 bytes from pnfjra-ak-in-f14.1e100.net (142.250.203.14): icmp_seq=24 ttl=118 time=2.68 ms
64 bytes from pnfjra-ak-in-f14.1e100.net (142.250.203.14): icmp_seq=25 ttl=118 time=2.73 ms
64 bytes from pnfjra-ak-in-f14.1e100.net (142.250.203.14): icmp_seq=26 ttl=118 time=2.77 ms
```

4. Stop, start, or terminate your instance (from CLI):

Stop

aws ec2 stop-instances --instance-ids <instance-id>

Start

aws ec2 start-instances --instance-ids <instance-id>

Terminate (optional)

aws ec2 terminate-instances --instance-ids <instance-id>

```
aws ec2 stop-instances --instance-ids i-03aca059942e31e5 i-024f5da3c0bbe497c
{
  "StoppingInstances": [
    {
      "InstanceId": "i-03aca059942e31e5",
      "CurrentState": {
        "Code": 64,
        "Name": "stopping"
      },
      "PreviousState": {
        "Code": 36,
        "Name": "running"
      }
    },
    {
      "InstanceId": "i-024f5da3c0bbe497c",
      "CurrentState": {
        "Code": 64,
        "Name": "stopping"
      },
      "PreviousState": {
        "Code": 36,
        "Name": "running"
      }
    }
  ]
}

aws ec2 start-instances --instance-ids i-03aca059942e31e5 i-024f5da3c0bbe497c
{
  "StartingInstances": [
    {
      "InstanceId": "i-024f5da3c0bbe497c",
      "CurrentState": {
        "Code": 0,
        "Name": "pending"
      },
      "PreviousState": {
        "Code": 80,
        "Name": "stopped"
      }
    },
    {
      "InstanceId": "i-03aca059942e31e5",
      "CurrentState": {
        "Code": 0,
        "Name": "pending"
      },
      "PreviousState": {
        "Code": 80,
        "Name": "stopped"
      }
    }
  ]
}
```

Task 5 — Understand AWS describe-* commands

1. Describe Security Groups

Command:

aws ec2 describe-security-groups

```
Windows PowerShell
023-22411-007-oss @ /workspaces/Lab09 (main) $ aws ec2 describe-security-groups
{
  "SecurityGroups": [
    {
      "GroupId": "sg-0aea5ee66ca09fe98",
      "IpPermissionsEgress": [
        {
          "IpProtocol": "-1",
          "UserIdGroupPairs": [],
          "IpRanges": [
            {
              "CidrIp": "0.0.0.0/0"
            }
          ],
          "Ipv6Ranges": [],
          "PrefixListIds": []
        }
      ],
      "VpcId": "vpc-082ef0dc0c9f1d07d",
      "SecurityGroupArn": "arn:aws:ec2:me-central-1:218496845154:security-group/sg-0aea5ee66ca09fe98",
      "OwnerId": "218496845154",
      "GroupName": "default",
      "Description": "default VPC security group",
      "IpPermissions": [
        {
          "IpProtocol": "tcp",
          "FromPort": 80,
          "ToPort": 80,
          "UserIdGroupPairs": [],
          "IpRanges": [
            {
              "CidrIp": "4.240.18.229/32"
            }
          ],
          "Ipv6Ranges": [],
          "PrefixListIds": []
        },
        {
          "IpProtocol": "-1",
          "UserIdGroupPairs": [
            {
              "UserId": "218496845154",
              "GroupId": "sg-0aea5ee66ca09fe98"
            }
          ]
        }
      ],
    }
  ]
}
```

2. Describe VPCs

Command:

aws ec2 describe-vpcs

```

    ],
023-22411-007-oss @ /workspaces/Lab09 (main) $ aws ec2 describe-vpcs
{
  "Vpcs": [
    {
      "OwnerId": "218496845154",
      "InstanceTenancy": "default",
      "CidrBlockAssociationSet": [
        {
          "AssociationId": "vpc-cidr-assoc-0742c8946b197a094",
          "CidrBlock": "172.31.0.0/16",
          "CidrBlockState": {
            "State": "associated"
          }
        }
      ],
      "IsDefault": true,
      "BlockPublicAccessStates": {
        "InternetGatewayBlockMode": "off"
      },
      "VpcId": "vpc-082ef0dc0c9f1d07d",
      "State": "available",
      "CidrBlock": "172.31.0.0/16",
      "DhcpOptionsId": "dopt-06f10cde7c4fc77fc"
    }
  ]
}

```

3. Describe Subnets

Command:

aws ec2 describe-subnets

```

023-22411-007-oss @ /workspaces/Lab09 (main) $ aws ec2 describe-subnets
{
  "Subnets": [
    {
      "AvailabilityZoneId": "mec1-az1",
      "MapCustomerOwnedIpOnLaunch": false,
      "OwnerId": "218496845154",
      "AssignIpv6AddressOnCreation": false,
      "Ipv6CidrBlockAssociationSet": [],
      "SubnetArn": "arn:aws:ec2:me-central-1:218496845154:subnet/subnet-0ed9f14bcd5dfebb3",
      "EnableDns64": false,
      "Ipv6Native": false,
      "PrivateDnsNameOptionsOnLaunch": {
        "HostnameType": "ip-name",
        "EnableResourceNameDnsARecord": false,
        "EnableResourceNameDnsAAAARecord": false
      },
      "BlockPublicAccessStates": {
        "InternetGatewayBlockMode": "off"
      },
      "SubnetId": "subnet-0ed9f14bcd5dfebb3",
      "State": "available",
      "VpcId": "vpc-082ef0dc0c9f1d07d",
      "CidrBlock": "172.31.0.0/24",
      "AvailableIpAddressCount": 249,
      "AvailabilityZone": "me-central-1a",
      "DefaultForAz": false,
      "MapPublicIpOnLaunch": true
    }
  ]
}
023-22411-007-oss @ /workspaces/Lab09 (main) $

```

4. Describe EC2 Instances

Command:

aws ec2 describe-instances

```
23-22411-007 oss @ /workspaces/Lab09 (main) $ aws ec2 describe-instances
{
  "Reservations": [
    {
      "ReservationId": "r-0e7560e94adff29ce",
      "OwnerId": "218496845154",
      "Groups": [],
      "Instances": [
        {
          "Architecture": "x86_64",
          "BlockDeviceMappings": [
            {
              "DeviceName": "/dev/xvda",
              "Ebs": {
                "AttachTime": "2026-01-23T10:05:46+00:00",
                "DeleteOnTermination": true,
                "Status": "attached",
                "VolumeId": "vol-0bb711cd52b4dbccf",
                "EbsCardIndex": 0
              }
            }
          ],
          "ClientToken": "1bab9298-b805-4978-ba61-5f0d00425a52",
          "EbsOptimized": false,
          "EnaSupport": true,
          "Hypervisor": "xen",
          "NetworkInterfaces": [
            {
              "Association": {
                "IpOwnerId": "amazon",
                "PublicDnsName": "ec2-3-29-124-86.me-central-1.compute.amazonaws.com",
                "PublicIp": "3.29.124.86"
              },
              "Attachment": {
                "AttachTime": "2026-01-23T10:05:45+00:00",
                "AttachmentId": "eni-attach-0069be1a421ad11dd",
                "DeleteOnTermination": true,
                "DeviceIndex": 0,
                "Status": "attached",
                "NetworkCardIndex": 0
              },
              "Description": "",
              "Groups": [
                {
                  "GroupId": "sg-0aea5ee66ca09fe98",

```

5. Describe Regions

Command:

aws ec2 describe-regions

```
@23-22411-007-oss @ /workspaces/Lab09 (main) $ aws ec2 describe-regions
{
  "Regions": [
    {
      "OptInStatus": "opt-in-not-required",
      "RegionName": "ap-south-1",
      "Endpoint": "ec2.ap-south-1.amazonaws.com"
    },
    {
      "OptInStatus": "opt-in-not-required",
      "RegionName": "eu-north-1",
      "Endpoint": "ec2.eu-north-1.amazonaws.com"
    },
    {
      "OptInStatus": "opt-in-not-required",
      "RegionName": "eu-west-3",
      "Endpoint": "ec2.eu-west-3.amazonaws.com"
    },
    {
      "OptInStatus": "opt-in-not-required",
      "RegionName": "eu-west-2",
      "Endpoint": "ec2.eu-west-2.amazonaws.com"
    },
    {
      "OptInStatus": "opt-in-not-required",
      "RegionName": "eu-west-1",
      "Endpoint": "ec2.eu-west-1.amazonaws.com"
    },
    {
      "OptInStatus": "opt-in-not-required",
      "RegionName": "ap-northeast-3",
      "Endpoint": "ec2.ap-northeast-3.amazonaws.com"
    },
    {
      "OptInStatus": "opt-in-not-required",
      "RegionName": "ap-northeast-2",
      "Endpoint": "ec2.ap-northeast-2.amazonaws.com"
    },
    {
      "OptInStatus": "opt-in-not-required",
      "RegionName": "ap-northeast-1",
      "Endpoint": "ec2.ap-northeast-1.amazonaws.com"
    },
    {
      "OptInStatus": "opted-in",
```

6. Describe Availability Zones

Command:

aws ec2 describe-availability-zones

```

    "OptInStatus": "opt-in-not-required",
@23-22411-007-oss [ ] /workspaces/Lab09 (main) $ aws ec2 describe-availability-zones
{
  "AvailabilityZones": [
    {
      "OptInStatus": "opt-in-not-required",
      "Messages": [],
      "RegionName": "me-central-1",
      "ZoneName": "me-central-1a",
      "ZoneId": "mec1-az1",
      "GroupName": "me-central-1-zg-1",
      "NetworkBorderGroup": "me-central-1",
      "ZoneType": "availability-zone",
      "GroupLongName": "Middle East (UAE) 1",
      "State": "available"
    },
    {
      "OptInStatus": "opt-in-not-required",
      "Messages": [],
      "RegionName": "me-central-1",
      "ZoneName": "me-central-1b",
      "ZoneId": "mec1-az2",
      "GroupName": "me-central-1-zg-1",
      "NetworkBorderGroup": "me-central-1",
      "ZoneType": "availability-zone",
      "GroupLongName": "Middle East (UAE) 1",
      "State": "available"
    },
    {
      "OptInStatus": "opt-in-not-required",
      "Messages": [],
      "RegionName": "me-central-1",
      "ZoneName": "me-central-1c",
      "ZoneId": "mec1-az3",
      "GroupName": "me-central-1-zg-1",
      "NetworkBorderGroup": "me-central-1",
      "ZoneType": "availability-zone",
      "GroupLongName": "Middle East (UAE) 1",
      "State": "available"
    }
  ]
}
@23-22411-007-oss [ ] /workspaces/Lab09 (main) $ █

```

Task 6 — IAM (CLI) Complete Step-by-Step Guide

◆ Prerequisites (DO THIS FIRST)

1. AWS CLI installed
2. You are logged in as an **IAM user or root** with IAM permissions
3. Run:

aws sts get-caller-identity

```

@23-22411-007-oss [ ] /workspaces/Lab09 (main) $ aws sts get-caller-identity
{
  "UserId": "AIDATFX3NIVRBKXYLMQ05",
  "Account": "218496845154",
  "Arn": "arn:aws:iam::218496845154:user/lab9"
}
@23-22411-007-oss [ ] /workspaces/Lab09 (main) $

```

▣ STEP 1: Create IAM Group

aws iam create-group --group-name MyGroupCli

```
@23-22411-007-oss [ /workspaces/Lab09 (main) ] $ aws iam create-group --group-name MyGroupCli
{
  "Group": {
    "Path": "/",
    "GroupName": "MyGroupCli",
    "GroupId": "AGPATFX3NIVRORROYKGXA5",
    "Arn": "arn:aws:iam::218496845154:group/MyGroupCli",
    "CreateDate": "2026-01-23T16:53:35+00:00"
  }
}
```

■ STEP 2: Get Group Details

aws iam get-group --group-name MyGroupCli

```
@23-22411-007-oss [ /workspaces/Lab09 (main) ] $ aws iam get-group --group-name MyGroupCli
{
  "Users": [],
  "Group": {
    "Path": "/",
    "GroupName": "MyGroupCli",
    "GroupId": "AGPATFX3NIVRORROYKGXA5",
    "Arn": "arn:aws:iam::218496845154:group/MyGroupCli",
    "CreateDate": "2026-01-23T16:53:35+00:00"
  }
}
```

■ STEP 3: Create IAM User

aws iam create-user --user-name MyUserCli

```
@23-22411-007-oss [ /workspaces/Lab09 (main) ] $ aws iam create-user --user-name MyUserCli
{
  "User": {
    "Path": "/",
    "UserName": "MyUserCli",
    "UserId": "AIDATFX3NIVREVSWCVKHH",
    "Arn": "arn:aws:iam::218496845154:user/MyUserCli",
    "CreateDate": "2026-01-23T16:54:51+00:00"
  }
}
```

■ STEP 4: Get User Details

aws iam get-user --user-name MyUserCli

```
@23-22411-007-oss @ /workspaces/Lab09 (main) $ aws iam get-user --user-name MyUserCli
{
  "User": {
    "Path": "/",
    "UserName": "MyUserCli",
    "UserId": "AIDATFX3NIVREVSWCVKHH",
    "Arn": "arn:aws:iam::218496845154:user/MyUserCli",
    "CreateDate": "2026-01-23T16:54:51+00:00"
  }
}
```

■ STEP 5: Add User to Group

```
aws iam add-user-to-group \
  --user-name MyUserCli \
  --group-name MyGroupCli
```

```
● @23-22411-007-oss → /workspaces/Lab09 (main) $ aws iam add-user-to-group \
  --user-name MyUserCli \
  --group-name MyGroupCli
○ @23-22411-007-oss → /workspaces/Lab09 (main) $ █
```

■ STEP 6: Verify User Is in Group

```
aws iam get-group --group-name MyGroupCli
```

```
@23-22411-007-oss @ /workspaces/Lab09 (main) $ aws iam get-group --group-name MyGroupCli
{
  "Users": [
    {
      "Path": "/",
      "UserName": "MyUserCli",
      "UserId": "AIDATFX3NIVREVSWCVKHH",
      "Arn": "arn:aws:iam::218496845154:user/MyUserCli",
      "CreateDate": "2026-01-23T16:54:51+00:00"
    }
  ],
  "Group": {
    "Path": "/",
    "GroupName": "MyGroupCli",
    "GroupId": "AGPATFX3NIVROROKGXA5",
    "Arn": "arn:aws:iam::218496845154:group/MyGroupCli",
    "CreateDate": "2026-01-23T16:53:35+00:00"
  }
}
```

■ STEP 7: List Policies That Mention EC2

```
aws iam list-policies \
  --query "Policies[?contains(PolicyName, 'EC2')].{Name:PolicyName}" \
  --output text
```

```
@23-22411-007-oss →/workspaces/Lab09 (main) $ aws iam list-policies \
--query "Policies[?contains(PolicyName, 'EC2')].{Name:PolicyName}" \
--output text
AmazonElasticMapReduceforEC2Role
AmazonEC2RoleforDataPipelineRole
AmazonEC2ContainerServiceforEC2Role
AmazonEC2ContainerServiceRole
AmazonEC2RoleforAWSCodeDeploy
AmazonEC2RoleforSSM
CloudWatchActionsEC2Access
AmazonEC2ContainerRegistryReadOnly
AmazonEC2ContainerRegistryPowerUser
AmazonEC2ContainerRegistryFullAccess
AmazonEC2ContainerServiceAutoscaleRole
AmazonEC2SpotFleetAutoscaleRole
AWSElasticBeanstalkCustomPlatformforEC2Role
AmazonEC2ContainerServiceEventsRole
AmazonEC2SpotFleetTaggingRole
AWSSEC2SpotServiceRolePolicy
AWSServiceRoleForEC2ScheduledInstances
AWSSEC2SpotFleetServiceRolePolicy
AWSApplicationAutoscalingEC2SpotFleetRequestPolicy
AWSSEC2FleetServiceRolePolicy
AWSAutoScalingPlansEC2AutoScalingPolicy
EC2InstanceConnect
AmazonEC2RolePolicyForLaunchWizard
EC2InstanceProfileForImageBuilder
EC2FleetTimeShiftableServiceRolePolicy
AmazonEC2RoleforAWSCodeDeployLimited
EC2InstanceProfileForImageBuilderECRContainerBuilds
```

▣ STEP 8: Get ARN for AmazonEC2FullAccess

aws iam list-policies \

```
--query 'Policies[?PolicyName==`AmazonEC2FullAccess`].{Name:PolicyName, ARN:Arn}' \
```

```
--output table
```

✓ Copy the ARN: arn:aws:iam::aws:policy/AmazonEC2FullAccess

```
@23-22411-007-oss →/workspaces/Lab09 (main) $ aws iam list-policies \
--query 'Policies[?PolicyName==`AmazonEC2FullAccess`].{Name:PolicyName, ARN:Arn}' \
--output table
-----
|                               ListPolicies                               |
+-----+-----+-----+-----+
|                               ARN                               | Name |
+-----+-----+-----+-----+
| arn:aws:iam::aws:policy/AmazonEC2FullAccess | AmazonEC2FullAccess |
+-----+-----+-----+-----+
@23-22411-007-oss →/workspaces/Lab09 (main) $
```

▣ STEP 9: Attach EC2 Policy to Group

aws iam attach-group-policy \

```
--group-name MyGroupCli \  
  
--policy- arn:aws:iam::aws:policy/AmazonEC2FullAccess
```

```
@23-22411-007-oss →/workspaces/Lab09 (main) $ aws iam attach-group-policy \  
  --group-name MyGroupCli \  
  --policy- arn:aws:iam::aws:policy/AmazonEC2FullAccess  
@23-22411-007-oss →/workspaces/Lab09 (main) $
```

■ STEP 10: List Attached Policies

```
aws iam list-attached-group-policies \  
  
--group-name MyGroupCli
```

```
@23-22411-007-oss →/workspaces/Lab09 (main) $ aws iam list-attached-group-policies \  
  --group-name MyGroupCli  
{  
  "AttachedPolicies": [  
    {  
      "PolicyName": "AmazonEC2FullAccess",  
      "PolicyArn": "arn:aws:iam::aws:policy/AmazonEC2FullAccess"  
    }  
  ]  
}
```

■ STEP 11: Create Console Login Profile

⚠ Choose a strong password (don't show it)

```
aws iam create-login-profile \  
  
--user-name MyUserCli \  
  
--password TempPass@123 \  
  
--password-reset-required
```

```
@23-22411-007-oss →/workspaces/Lab09 (main) $ aws iam create-login-profile \  
  --user-name MyUserCli \  
  --password TempPass@123 \  
  --password-reset-required  
{  
  "LoginProfile": {  
    "UserName": "MyUserCli",  
    "CreateDate": "2026-01-23T17:00:09+00:00",  
    "PasswordResetRequired": true  
  }  
}
```

■ STEP 12: Allow User to Change Password (Temporary)

```
aws iam attach-group-policy \  
  
--group-name MyGroupCli \  
  
--policy- arn:aws:iam::aws:policy/AmazonEC2FullAccess
```

```
--policy-arn arn:aws:iam::aws:policy/IAMUserChangePassword
```

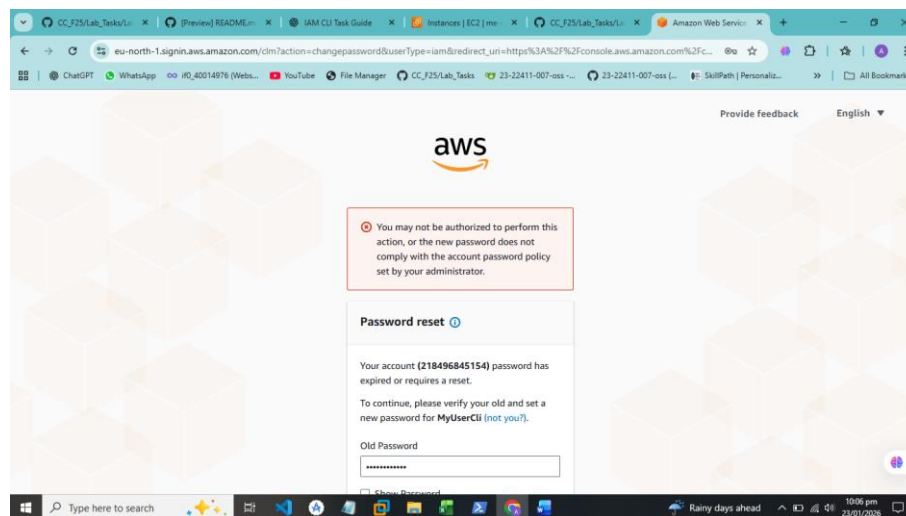
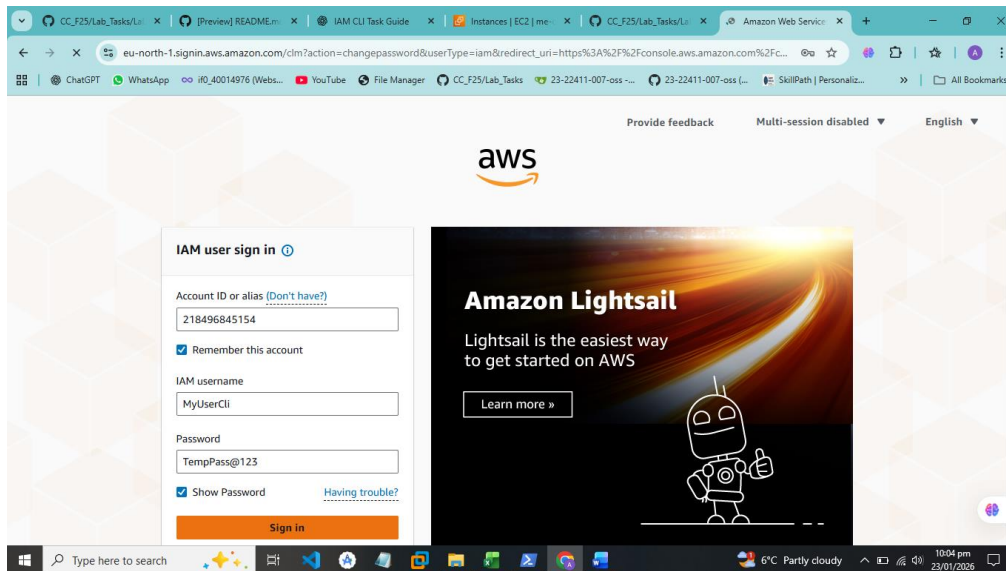
```
@23-22411-007-oss →/workspaces/Lab09 (main) $ aws iam attach-group-policy \  
--group-name MyGroupCli \  
--policy-arn arn:aws:iam::aws:policy/IAMUserChangePassword  
@23-22411-007-oss →/workspaces/Lab09 (main) $
```

User Login (Console)

1. Sign in URL:

https://<ACCOUNT_ID>.signin.aws.amazon.com/console

2. Username: MyUserCli
3. Change password



■ STEP 13: Detach Change Password Policy

aws iam detach-group-policy \

--group-name MyGroupCli \

--policy-arn arn:aws:iam::aws:policy/IAMUserChangePassword

```
@23-22411-007-oss →/workspaces/Lab09 (main) $ aws iam detach-group-policy \
--group-name MyGroupCli \
--policy-arn arn:aws:iam::aws:policy/IAMUserChangePassword
@23-22411-007-oss →/workspaces/Lab09 (main) $
```

■ STEP 14: Create Access Keys

aws iam create-access-key --user-name MyUserCli

```
@23-22411-007-oss →/workspaces/Lab09 (main) $ aws iam create-access-key --user-name MyUserCli
{
  "AccessKey": {
    "UserName": "MyUserCli",
    "AccessKeyId": "AKIATFX3NIVRONQUJREX",
    "Status": "Active",
    "SecretAccessKey": "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",
    "CreateDate": "2026-01-23T17:07:37+00:00"
  }
}
```

■ STEP 15: List Access Keys

aws iam list-access-keys --user-name MyUserCli

```
@23-22411-007-oss →/workspaces/Lab09 (main) $ aws iam list-access-keys --user-name MyUserCli
{
  "AccessKeyMetadata": [
    {
      "UserName": "MyUserCli",
      "AccessKeyId": "AKIATFX3NIVRONQUJREX",
      "Status": "Active",
      "CreateDate": "2026-01-23T17:07:37+00:00"
    }
  ]
}
```

■ STEP 16: Authenticate Using Environment Variables

export AWS_ACCESS_KEY_ID=AKIA...

export AWS_SECRET_ACCESS_KEY=xxxxxxxx

printenv | grep AWS_

```
@23-22411-007-oss →/workspaces/Lab09 (main) $ export AWS_ACCESS_KEY_ID=AKIA...
export AWS_SECRET_ACCESS_KEY=xxxxxxx
printenv | grep AWS_
AWS_SECRET_ACCESS_KEY=xxxxxxx
AWS_ACCESS_KEY_ID=AKIA...
```

▣ STEP 17: Test Permission (Expected Failure)

aws iam get-user --user-name MyUserCli

```
@23-22411-007-oss →/workspaces/Lab09 (main) $ aws iam get-user --user-name MyUserCli
An error occurred (InvalidClientTokenId) when calling the GetUser operation: The security token included in the request is invalid.
@23-22411-007-oss →/workspaces/Lab09 (main) $
```

▣ STEP 18: Clear Credentials

exit

▣ STEP 19: Test Again With Admin Credentials

aws iam get-user --user-name MyUserCli

```
@23-22411-007-oss →/workspaces/Lab09 (main) $ aws iam get-user --user-name MyUserCli
{
  "User": {
    "Path": "/",
    "UserName": "MyUserCli",
    "UserId": "AIDATFX3NIVREVSVCVHH",
    "Arn": "arn:aws:iam::218496845154:user/MyUserCli",
    "CreateDate": "2026-01-23T16:54:51+00:00",
    "PasswordLastUsed": "2026-01-23T17:04:28+00:00"
  }
}
```

Task 7 — Filters & Queries (Step-by-Step)

🔗 1 Filter by tag name → get Public IP

This finds instances whose **Name tag = MyServer** and prints **Public IP only**.

aws ec2 describe-instances \

--filters "Name=tag:Name,Values=MyServer" \

--query "Reservations[*].Instances[*].PublicIpAddress" \

--output text

```

• @23-22411-007-oss →/workspaces/Lab09 (main) $ aws ec2 describe-instances \
  --filters "Name=tag:Name,Values=MyServer" \
  --query "Reservations[*].Instances[*].PublicIpAddress" \
  --output text
3.29.124.86
3.29.93.20
○ @23-22411-007-oss →/workspaces/Lab09 (main) $ █

```

2 Filter by instance type (t3.micro)

aws ec2 describe-instances \

--filters "Name=instance-type,Values=t3.micro" \

--query "Reservations[].Instances[].InstanceId" \

--output table

```

@23-22411-007-oss →/workspaces/Lab09 (main) $ aws ec2 describe-instances \
  --filters "Name=instance-type,Values=t3.micro" \
  --query "Reservations[].Instances[].InstanceId" \
  --output table
-----+-----
| DescribeInstances |
+-----+-----+
| i-024f5da3cddb497c |
| i-03aca059942e3c1e5 |
+-----+-----+
@23-22411-007-oss →/workspaces/Lab09 (main) $ █

```

3 Filter by subnet ID

Use **exact subnet ID** given in task:

aws ec2 describe-instances \

--filters "Name=subnet-id,Values=subnet-0600df5fa8ce60857" \

--query "Reservations[*].Instances[*].InstanceId" \

--output table

 If empty:

- Your instance might be in a **different subnet**
- You can list subnets:

aws ec2 describe-subnets --query "Subnets[].SubnetId"

```
@23-22411-007-oss →/workspaces/Lab09 (main) $ aws ec2 describe-instances \
--filters "Name=subnet-id,Values=subnet-0600df5fa8ce60857" \
--query "Reservations[*].Instances[*].InstanceId" \
--output table
@23-22411-007-oss →/workspaces/Lab09 (main) $ aws ec2 describe-subnets --query "Subnets[].SubnetId"
[
  "subnet-0ed9f14bcd5dfebb3"
]
@23-22411-007-oss →/workspaces/Lab09 (main) $
```

4 Filter by VPC ID

Use **exact VPC ID**:

```
aws ec2 describe-instances \
--filters "Name=vpc-id,Values=vpc-06be85cd81b657192" \
--query "Reservations[*].Instances[*].InstanceId" \
--output table
```

```
@23-22411-007-oss →/workspaces/Lab09 (main) $ aws ec2 describe-instances \
--filters "Name=vpc-id,Values=vpc-06be85cd81b657192" \
--query "Reservations[*].Instances[*].InstanceId" \
--output table
@23-22411-007-oss →/workspaces/Lab09 (main) $
```

Task 8 — Use --query to format outputs for reporting

Command 1: Instance ID, Public IP, Name (TABLE)

Run this command:

```
aws ec2 describe-instances \
--filters "Name=tag:Name,Values=MyServer" \
--query
"Reservations[*].Instances[*].[InstanceId,PublicIpAddress,Tags[?Key=='Name'].Value[0]]" \
--output table
```

```
@23-22411-007-oss →/workspaces/Lab09 (main) $ aws ec2 describe-instances \
--filters "Name=tag:Name,Values=MyServer" \
--query "Reservations[*].Instances[*].[InstanceId,PublicIpAddress,Tags[?Key=='Name'].Value|[0]]" \
--output table
```

DescribeInstances		
i-024f5da3cddb497c	3.29.124.86	MyServer
i-03aca059942e3c1e5	3.29.93.20	MyServer

```
@23-22411-007-oss →/workspaces/Lab09 (main) $
```

◆ Command 2: Instance ID and State (TABLE)

Run:

```
aws ec2 describe-instances \
```

```
--query "Reservations[*].Instances[*].[InstanceId,State.Name]" \
```

```
--output table
```

```
@23-22411-007-oss →/workspaces/Lab09 (main) $ aws ec2 describe-instances \
--query "Reservations[*].Instances[*].[InstanceId,State.Name]" \
--output table
```

DescribeInstances	
i-024f5da3cddb497c	running
i-03aca059942e3c1e5	running

```
@23-22411-007-oss →/workspaces/Lab09 (main) $
```

◆ Command 3: Instance ID, Type, Availability Zone (TABLE)

Run:

```
aws ec2 describe-instances \
```

```
--query "Reservations[*].Instances[*].[InstanceId,InstanceType,Placement.AvailabilityZone]" \
```

```
--output table
```

```
@23-22411-007-oss →/workspaces/Lab09 (main) $ aws ec2 describe-instances \
--query "Reservations[*].Instances[*].[InstanceId,InstanceType,Placement.AvailabilityZone]" \
--output table
```

DescribeInstances		
i-024f5da3cddb497c	t3.micro	me-central-1a
i-03aca059942e3c1e5	t3.micro	me-central-1a

```
@23-22411-007-oss →/workspaces/Lab09 (main) $
```

Cleanup — Remove resources to avoid charges

1 Terminate EC2 Instances

Step

First, list instances to get the real instance ID (don't use EXAMPLE).

`aws ec2 describe-instances --query "Reservations[].Instances[].InstanceId" --output table`

```
@23-22411-007-oss →/workspaces/Lab09 (main) $ aws ec2 describe-instances --query "Reservations[].Instances[].InstanceId" --output table
-----
| DescribeInstances |
+-----+
| i-024f5da3cddb497c |
| i-03aca059942e3c1e5 |
+-----+
```

Now terminate the instance:

`aws ec2 terminate-instances --instance-ids i-XXXXXXXXXXXXX`

```
@23-22411-007-oss →/workspaces/Lab09 (main) $ aws ec2 terminate-instances --instance-ids i-024f5da3cddb497c i-03aca059942e3c1e5
{
  "TerminatingInstances": [
    {
      "InstanceId": "i-03aca059942e3c1e5",
      "CurrentState": {
        "Code": 32,
        "Name": "shutting-down"
      },
      "PreviousState": {
        "Code": 16,
        "Name": "running"
      }
    },
    {
      "InstanceId": "i-024f5da3cddb497c",
      "CurrentState": {
        "Code": 32,
        "Name": "shutting-down"
      },
      "PreviousState": {
        "Code": 16,
        "Name": "running"
      }
    }
  ]
}
```

[2] Delete EBS Volumes & Snapshots (if any)

List volumes

`aws ec2 describe-volumes --query "Volumes[].VolumeId" --output table`

If volumes exist, delete them:

`aws ec2 delete-volume --volume-id vol-XXXXXXXXX`

```
@23-22411-007-oss →/workspaces/Lab09 (main) $ aws ec2 describe-volumes --query "Volumes[].VolumeId" --output table
-----
| DescribeVolumes |
+-----+
| vol-0f209e3216db8b109 |
+-----+

@23-22411-007-oss →/workspaces/Lab09 (main) $ aws ec2 delete-volume --volume-id vol-0f209e3216db8b109

An error occurred (InvalidVolume.NotFound) when calling the DeleteVolume operation: The volume 'vol-0f209e3216db8b109' does not exist.
```

List snapshots

`aws ec2 describe-snapshots --owner-ids self --query "Snapshots[].SnapshotId" --output table`

```
@23-22411-007-oss →/workspaces/Lab09 (main) $ aws ec2 describe-snapshots --owner-ids self --query "Snapshots[].SnapshotId" --output table
@23-22411-007-oss →/workspaces/Lab09 (main) $
```

3 Delete Security Group & Key Pair

List security groups (to confirm ID)

```
aws ec2 describe-security-groups --query "SecurityGroups[].GroupId" --output table
```

Delete security group:

```
aws ec2 delete-security-group --group-id sg-XXXXXXX
```

Delete key pair:

```
aws ec2 delete-key-pair --key-name MyED25519Key
```

```
@23-22411-007-oss →/workspaces/Lab09 (main) $ aws ec2 describe-security-groups --query "SecurityGroups[].GroupId" --output table
+-----+
| DescribeSecurityGroups |
+-----+
| sg-0aea5ee66ca09fe98 |
+-----+
@23-22411-007-oss →/workspaces/Lab09 (main) $ aws ec2 delete-security-group --group-id sg-0aea5ee66ca09fe98

An error occurred (CannotDelete) when calling the DeleteSecurityGroup operation: the specified group: "sg-0aea5ee66ca09fe98" name: "default" cannot be deleted by a user
@23-22411-007-oss →/workspaces/Lab09 (main) $ aws ec2 delete-key-pair --key-name MyED25519Key
{
  "Return": true,
  "KeyPairId": "key-0aadfd612956f55ad"
}
@23-22411-007-oss →/workspaces/Lab09 (main) $
```

4 Remove IAM User, Keys, and Group

List access keys

```
aws iam list-access-keys --user-name MyUserCli
```

Delete access key:

```
aws iam delete-access-key \
```

```
--user-name MyUserCli \
```

```
--access-key-id AKIAXXXXXXXX
```

Delete console login:

```
aws iam delete-login-profile --user-name MyUserCli
```

Remove user from group:

```
aws iam remove-user-from-group \
```

```
--user-name MyUserCli \
```

--group-name MyGroupCli

Delete user:

aws iam delete-user --user-name MyUserCli

Detach policy from group:

aws iam detach-group-policy \

--group-name MyGroupCli \

--policy-arn arn:aws:iam::aws:policy/EXAMPLEPolicyName

Delete group:

aws iam delete-group --group-name MyGroupCli

```
@23-22411-007-oss →/workspaces/Lab09 (main) $ aws iam delete-login-profile --user-name MyUserCli
@23-22411-007-oss →/workspaces/Lab09 (main) $ aws iam remove-user-from-group \
--user-name MyUserCli \
--group-name MyGroupCli
@23-22411-007-oss →/workspaces/Lab09 (main) $ aws iam delete-user --user-name MyUserCli
@23-22411-007-oss →/workspaces/Lab09 (main) $ aws iam detach-group-policy \
--group-name MyGroupCli \
--policy-arn arn:aws:iam::aws:policy/EXAMPLEPolicyName

An error occurred (NoSuchEntity) when calling the DetachGroupPolicy operation: Policy arn:aws:iam::aws:policy/EXAMPLEPolicyName was not found.
@23-22411-007-oss →/workspaces/Lab09 (main) $ aws iam detach-group-policy \
--group-name MyGroupCli \
--policy-arn arn:aws:iam::aws:policy/IAMUserChangePassword

An error occurred (NoSuchEntity) when calling the DetachGroupPolicy operation: Policy arn:aws:iam::aws:policy/IAMUserChangePassword was not found.
@23-22411-007-oss →/workspaces/Lab09 (main) $ aws iam list-attached-group-policies --group-name MyGroupCli
{
  "AttachedPolicies": [
    {
      "PolicyName": "AmazonEC2FullAccess",
      "PolicyArn": "arn:aws:iam::aws:policy/AmazonEC2FullAccess"
    }
  ]
}
@23-22411-007-oss →/workspaces/Lab09 (main) $ aws iam detach-group-policy --group-name MyGroupCli --policy-arn arn:aws:iam::aws:policy/AmazonEC2FullAccess
@23-22411-007-oss →/workspaces/Lab09 (main) $ aws iam delete-group --group-name MyGroupCli
@23-22411-007-oss →/workspaces/Lab09 (main) $
```

[5] Final Verification (Very Important)

CLI checks (optional but good)

aws ec2 describe-instances

aws iam list-users

aws ec2 describe-volumes

```
@23-22411-007-oss → /workspaces/Lab09 (main) $ aws ec2 describe-instances
{
  "SecurityGroups": [],
  "StateReason": {
    "Code": "Client.UserInitiatedShutdown",
    "Message": "Client.UserInitiatedShutdown: User initiated shutdown"
  },
  "Tags": [
    {
      "Key": "Name",
      "Value": "MyServer"
    }
  ],
  "VirtualizationType": "hvm",
@23-22411-007-oss → /workspaces/Lab09 (main) $ aws iam list-users
{
  "Users": [
    {
      "Path": "/",
      "UserName": "lab9",
      "UserId": "AIDATFX3NIVRBKXYLMQ05",
      "Arn": "arn:aws:iam::218496845154:user/lab9",
      "CreateDate": "2026-01-23T08:54:12+00:00"
    }
  ]
}
@23-22411-007-oss → /workspaces/Lab09 (main) $ aws ec2 describe-volumes
{
  "Volumes": []
}
```

AWS Console check (recommended)

1. Open AWS Console
2. Go to:
 - EC2 → Instances → Running (0)
 - EC2 → Volumes → No volumes
 - IAM → Users → Empty
3. Billing → No active services

